



南開大學
Nankai University

计算机学院

深度学习及应用（高阶课）实验报告

注意力机制算法

姓名：申健强

学号：2313119

专业：计算机科学与技术

2026 年 6 月 24 日

目录

1 实验目的与要求	2
2 数据集与实验设置	2
3 模型结构与原理	2
3.1 编码器 EncoderRNN	2
3.2 简单解码器 DecoderRNN (无注意力)	2
3.3 注意力解码器 AttnDecoderRNN (Bahdanau 注意力)	3
4 实验结果	3
5 对比分析: 简单 Seq2Seq vs 注意力 Seq2Seq	6
6 实验心得 (重点): 注意力机制的影响	6
7 结论	7

1 实验目的与要求

本实验的目标是掌握注意力机制的基本原理，并使用 PyTorch 搭建 Seq2Seq 模型完成法语 → 英语的翻译任务。具体要求：

- 实现基于 RNN 解码器的 Seq2Seq 模型（无注意力），查看其网络结构、训练损失与翻译结果；
- 实现基于**注意力机制**的 Seq2Seq 模型完成翻译，并与上面的纯 RNN Seq2Seq 模型**对比结果**；
- 重点阐述实验心得：加入注意力机制后，模型对实验结果的影响。

2 数据集与实验设置

数据集. 采用 PyTorch 官方的英法翻译语料 `eng-fra`，原始约 13.5 万句对。按教程做法过滤：源/目标句长均小于 10 个词，且英文以”i am / he is / she is / you are / we are / they are” 等前缀开头，最终得到 **11445** 个句对。建立词表后法语 4601 词、英语 2991 词，每个词用其索引表示（词级 one-hot）。

训练设置. 编码器与解码器隐藏维度均为 128，批大小 32，优化器为 Adam（学习率 0.001），损失函数为 NLLLoss（解码端输出经 LogSoftmax），训练采用 **teacher forcing**，最大解码长度 10，各训练 30 个 epoch（GPU）。两套模型（简单 vs 注意力）使用相同设置以保证可比，并记录每个 epoch 的训练损失以绘制曲线。模型定义在 `models.py`，数据/训练/评估/BLEU/绘图在 `train_utils.py`。

3 模型结构与原理

Seq2Seq 由**编码器**和**解码器**两个 RNN 组成：编码器读入源句得到隐藏表示，解码器据此逐词生成目标句。

3.1 编码器 EncoderRNN

编码器是一个 GRU，对每个源词嵌入后送入 GRU，输出每个时间步的隐藏状态序列 $\{h_i\}$ 与最终隐藏态。

3.2 简单解码器 DecoderRNN（无注意力）

最简单的解码器仅使用编码器的**最后一个隐藏态**作为唯一的上下文向量（context vector）来初始化解码，之后每步用上一步输出作为输入。其单步实现如下：

```
1 def forward_step(self, input, hidden):      # simple decoder step
2     output = self.embedding(input)
3     output = F.relu(output)
4     output, hidden = self.gru(output, hidden)
5     output = self.out(output)
6     return output, hidden
```

其局限在于：整句信息被压缩进一个固定长度向量，长句容易丢失信息，且解码时无法回看源端的具体位置。

3.3 注意力解码器 AttnDecoderRNN (Bahdanau 注意力)

注意力机制让解码器在生成**每个词时**动态地对所有编码器输出 $\{h_i\}$ 加权，从而“聚焦”到相关的源词。本实验采用 Bahdanau（加性）注意力，设解码器上一隐藏态为 s_{t-1} ：

$$e_{ti} = V_a^\top \tanh(W_a s_{t-1} + U_a h_i), \quad \alpha_{ti} = \frac{\exp(e_{ti})}{\sum_j \exp(e_{tj})}, \quad c_t = \sum_i \alpha_{ti} h_i. \quad (1)$$

即先对每个源位置打分、softmax 归一化为注意力权重 α_{ti} ，再加权求和得到该步的上下文 c_t 。注意力与解码单步的实现如下：

```

1 def forward(self, query, keys):                # Bahdanau additive attention
2     scores = self.Va(torch.tanh(self.Wa(query) + self.Ua(keys))) # (B, seq, 1)
3     scores = scores.squeeze(2).unsqueeze(1)    # (B, 1, seq)
4     weights = F.softmax(scores, dim=-1)        # attention weights
5     context = torch.bmm(weights, keys)         # (B, 1, hidden)
6     return context, weights
7
8 def forward_step(self, input, hidden, encoder_outputs):
9     embedded = self.dropout(self.embedding(input))
10    query = hidden.permute(1, 0, 2)
11    context, attn_weights = self.attention(query, encoder_outputs)
12    input_gru = torch.cat((embedded, context), dim=2) # concat context + embedding
13    output, hidden = self.gru(input_gru, hidden)
14    output = self.out(output)
15    return output, hidden, attn_weights

```

4 实验结果

两套模型在相同设置下各训练 30 个 epoch，总体结果见表 1。注意力模型在翻译质量（BLEU）上大幅领先：**86.68** 对 56.20，且最终训练损失更低。

表 1: 简单 Seq2Seq 与注意力 Seq2Seq 结果对比

模型	参数量	最终训练损失	BLEU	训练耗时 (min)
简单 Seq2Seq (无注意力)	1,555,759	0.197	56.20	5.6
注意力 Seq2Seq (Bahdanau)	1,638,064	0.075	86.68	11.5

训练损失曲线。图 4.1 为两套模型的训练损失随 epoch 的变化，注意力模型收敛更快、最终损失更低。

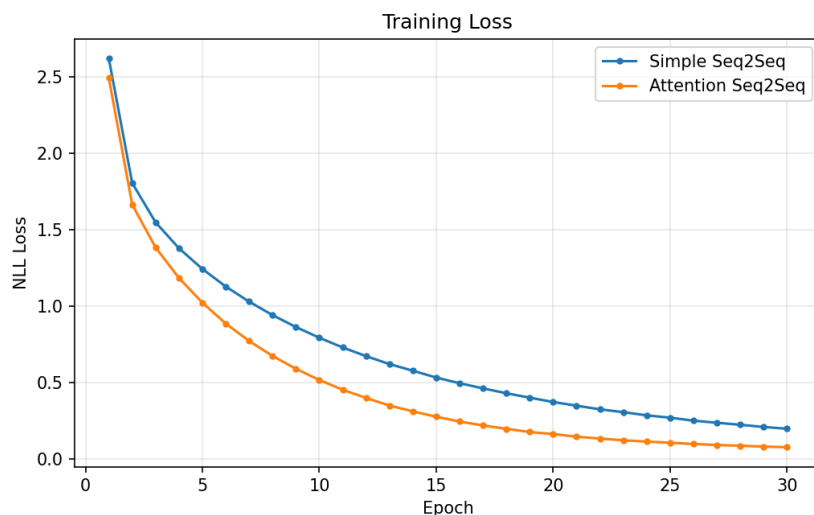


图 4.1: 训练损失曲线 (简单 vs 注意力 Seq2Seq)

翻译结果对比. 表 2 给出若干测试样例。可见简单模型常出现语义偏差或乱译（如把“迟到”译成“比赛进行到一半”、把“迷人”译成“性格外向”），而注意力模型几乎逐句命中参考译文。

表 2: 翻译样例对比 (源 / 参考 / 简单 / 注意力)

源 (法语)	参考 (英语)	简单 Seq2Seq	注意力 Seq2Seq
nous prenons du retard	we re running late	we re halfway in the race	we re running late
elle est fiere de ses eleves	she is proud of her students	she is proud of her son	she is proud of her students
vous etes charmant	you re charming	you re extroverted	you re charming
je le fais pour lui	i m doing this for him	i m doing this for the right thing	i m doing this for him
je suis desole de la reponse tardive	i m sorry for the late response	i m sorry for the being blackmailed by this late	i m sorry for the late response

注意力可视化. 图 4.2 给出注意力模型在 6 个测试样例上的注意力热力图（每行是一个英文输出词对全部法语源词的注意力分布）。可观察到三类典型现象：(1) **单调对齐**，如“discuter” → “talking”、“riches” → “rich”；(2) **否定结构对齐**，法语“ne...pas”的“pas”被“not”准确关注（如“je ne suis pas decourage” → “i m not discouraged”）；(3) **语序重排**，法语“il perd tout le temps son parapluie”译为“he is always losing his umbrella”时，“always”关注“tout le temps”、“his”关注“son”、“umbrella”关注“parapluie”，注意力随译序在源端**跳跃对齐**。这正是注意力可解释性的直接体现。

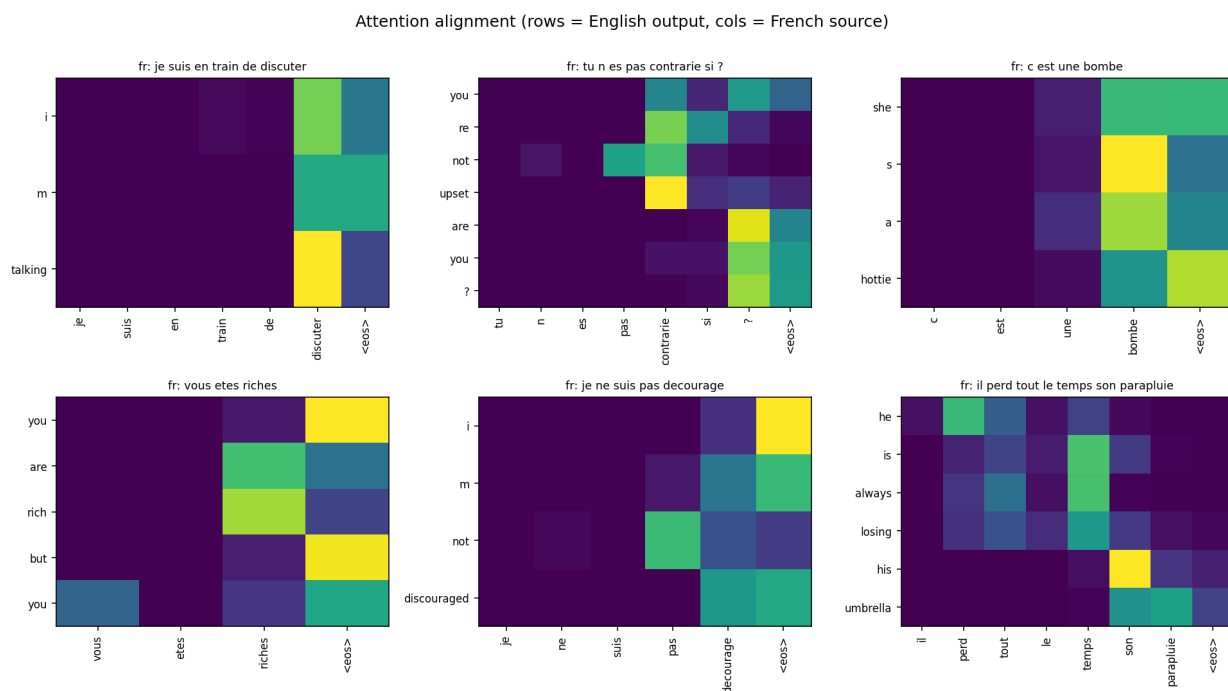


图 4.2: 注意力对齐热力图 (6 例): 行 = 英文输出词, 列 = 法语源词 (含 `<eos>`)。涵盖单调对齐、否定 (`pas`→`not`) 与语序重排 (如 `parapluie`→`umbrella`) 等情形。

解码的 token 级输出. 图 4.3 展示注意力模型在每个输出步的 top-5 候选英文词及其概率: top-1 行从左到右读即为译文, 颜色深浅反映该步的把握程度。可见模型在确定的词 (如 “he/is/losing/umbrella”) 上概率高度集中, 而在同义可替换处 (如 “hottie/beauty/cutie/singer”) 把概率分散到多个合理候选——直观呈现了序列生成**逐词决策**的过程。

Decoding trace: top-5 candidate English words at each output step (top-1 row read left-to-right = the translation)

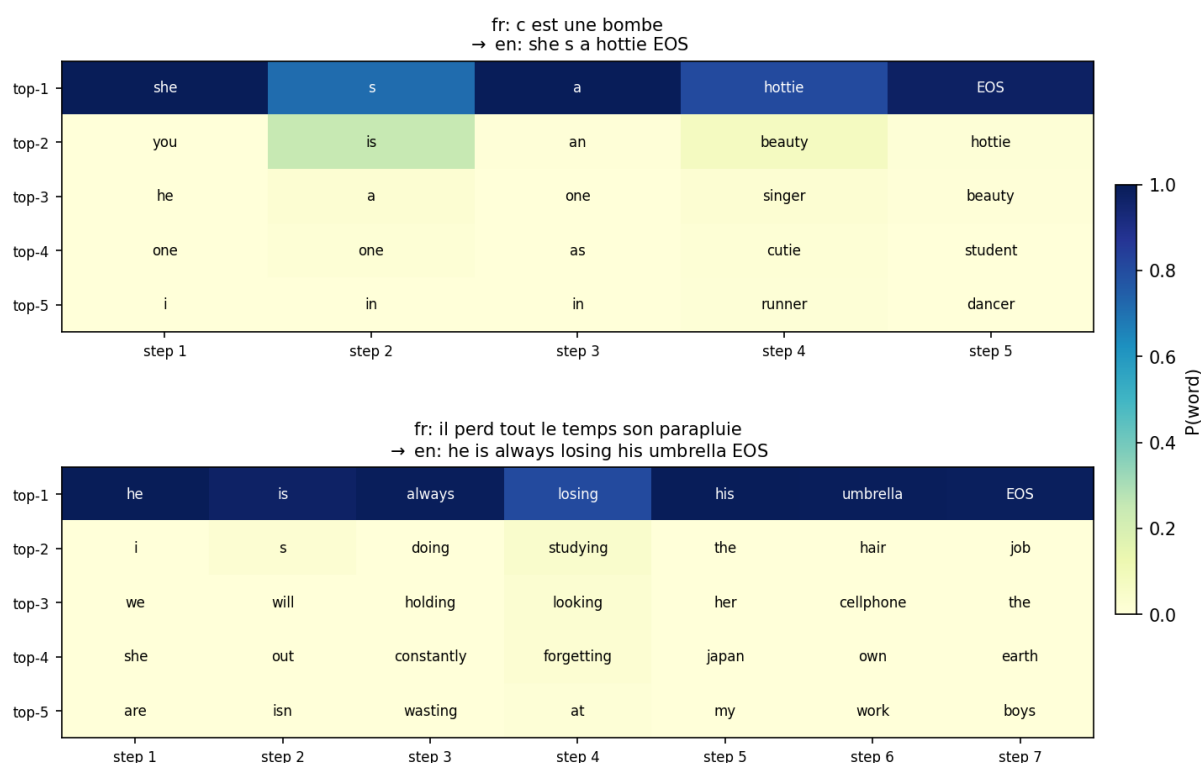


图 4.3: 解码轨迹：每个输出步的 top-5 候选词与概率（top-1 行即译文）。确定处概率集中、同义处分散到多个合理候选。

5 对比分析：简单 Seq2Seq vs 注意力 Seq2Seq

两套模型结构相近、参数量相当（约 1.56M 对 1.64M），唯一关键差别是解码端是否使用注意力，但结果差距显著：

- **翻译质量**：BLEU 从 56.20 提升到 86.68 (+30)。从样例看，简单模型在词义选择上容易跑偏，常生成“看似流畅但语义错误”的句子；注意力模型则能准确还原源句含义。
- **收敛与损失**：注意力模型最终训练损失（0.075）明显低于简单模型（0.197），收敛曲线也更低（图 4.1）。
- **代价**：注意力解码每步都要对源序列计算一次权重，训练时间约为简单模型的两倍（11.5 min 对 5.6 min），但带来的质量提升远超这一开销。

6 实验心得（重点）：注意力机制的影响

本次实验最直接的体会是：注意力机制有效缓解了 Seq2Seq 的信息瓶颈问题。

- **打破固定上下文的瓶颈**：简单解码器把整句压缩进编码器最后一个隐藏态这一单一固定向量，源句越长、信息损失越严重，解码时也无法回看源端细节；注意力让解码器在每个时间步都能按需读取整段编码器输出，相关源词获得更高权重，信息不再被迫挤进一个向量。

- **动态对齐与可解释性**：注意力权重天然给出了源词与译词的**软对齐**（图 4.2），不仅提升翻译质量，也让模型行为可解释——这是简单 Seq2Seq 不具备的。
- **结果印证**：在相同数据与训练设置下，仅引入注意力就把 BLEU 从 56 提升到 87，并显著降低损失，说明对翻译这类需要源-目标对齐的任务，注意力是**关键且高性价比**的改进。

长度维度的量化证据，进一步把测试句按源句长度分桶、分别计算两模型的 BLEU（图 6.4）：无注意力模型受**固定上下文向量**所限，BLEU 在各长度上都被压在 45 ~ 55 附近；而注意力模型在**所有长度**上都获得 +24 ~ 37 的大幅提升，并能随句子变长把 BLEU 抬升到 90 以上。这把“注意力打破信息瓶颈”从定性论述变成了可测量的结论。

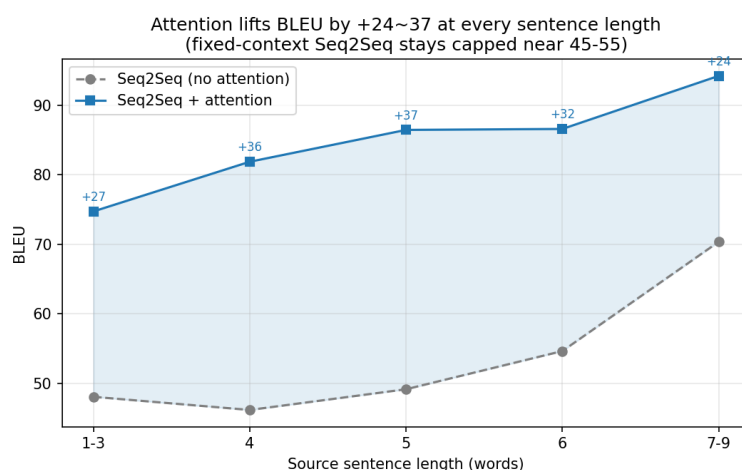


图 6.4: BLEU 随源句长度的变化：无注意力模型被固定上下文压在 45 ~ 55，注意力模型在各长度上均大幅领先。

7 结论

本实验实现并对比了无注意力的简单 Seq2Seq 与基于 Bahdanau 注意力的 Seq2Seq 在法英翻译上的表现。在结构相近、参数量相当的前提下，注意力模型把 BLEU 从 56.20 提升到 86.68、最终训练损失从 0.197 降到 0.075，翻译样例几乎逐句命中参考译文，注意力热力图也清晰展示了源-目标的对齐关系。结果表明：注意力机制通过在每个解码步动态聚焦相关源词，有效突破了固定上下文向量的信息瓶颈，是 Seq2Seq 翻译中关键的性能提升手段，代价仅为适度增加的计算开销。

参考文献

- [1] I. Sutskever, O. Vinyals, Q. V. Le. Sequence to Sequence Learning with Neural Networks. *NeurIPS*, 2014.
- [2] D. Bahdanau, K. Cho, Y. Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR*, 2015.
- [3] K. Cho et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP*, 2014.

-
- [4] S. Robertson. NLP From Scratch: Translation with a Sequence to Sequence Network and Attention. PyTorch Tutorials.